

```
7 {
8   while(atomic_CAS(&lock->val, 0, 1) != 0) {
9       atomic_FAA(&lock->waiters, 1);
10      futex_wait(&lock->val, 1);
11      atomic_FAA(&lock->waiters, -1);
12  }
13 }
14
15 void unlock(struct lock *lock)
16 {
17     lock->val = 0;
18     if (lock->waiters != 0)
19         futex_wake(&lock->val);
20 }
```

下面我们讨论在使用 futex 实现的互斥锁中是否存在丢失唤醒的现象。比如有线程在调用 futex_wait，但还未将线程插入内核的等待队列时，另一个线程同时调用 futex_wake，发现等待队列为空，就有可能造成丢失唤醒。为了避免这种情况，不同 futex 操作均使用互斥锁进行保护（如代码片段 10.13 中所示），因此多个 futex 操作之间只可能先后发生，不可能同时发生。因此在上面这种情况中：

1. 如果 futex_wait 操作先发生（代码片段 10.14 的第 10 行），即成功进入内核并拿到该 futex 的互斥锁（代码片段 10.13 的第 11 行），那么内核能保证该线程在另一个线程调用 futex_wake 并拿到互斥锁之前：

（a）要么回到用户态（lock->val 已经更新为 0，即另一个线程执行了代码片段 10.14 的第 17 行）。此时 futex_wait 操作发现值更改后直接放锁并返回（代码片段 10.13 中的第 19 行）。

（b）要么进入等待队列（lock->val 还未更新）。此时 futex_wait 先将当前线程加入等待队列中再释放互斥锁（代码片段 10.13 的第 14 行）。之后 futex_wake 获取互斥锁时会发现该线程（代码片段 10.13 的第 30 行）并唤醒。

2. 如果 futex_wake（代码片段 10.14 的第 19 行）操作先发生，即其先拿到 futex 的互斥锁（代码片段 10.13 的第 29 行），则 lock->val 一定已经更新为 0，因此后续的 futex_wait 在拿到互斥锁后会立即返回用户态。

所以在任何情况下，调用 futex_wait 的线程都不会由于错过 futex_wake 而陷入无限等待并造成丢失唤醒。

代码片段 10.15 展示了如何使用 futex 实现条件变量的两个操作。这里提供了简化的实现，其中 cond_signal 只能在与之搭配锁的临界区内被调用。条件变量提供了一个成员变量 value 作为 futex 操作的对象。cond_wait 操作将先读取 value 的旧值并将其存到局部变量 local 中（第 7 行），然后释放互斥锁（第 8 行），之后利用 futex_wait 操作挂起（第 9 行）。唤醒后，该线程需要重新获取互斥锁（第 10 行）。而 cond_

signal 操作则先更新 value 的值，然后调用 `futex_wake` 唤醒挂起的线程。该实现也不存在丢失唤醒的情况：如果执行 `cond_wait` 操作的线程在放锁之后（第 8 行）还未执行 `futex_wait`（第 9 行）时，有其他线程进入临界区并调用了 `futex_wake`（第 16 行），那么条件变量的 value 也一定被更新（第 15 行），因此 `futex_wait` 会发现该值与本地存储的旧值（`local`）不相等，将立即返回。否则，该线程将挂起并等待后续线程唤醒。因此，这里将条件变量“原子放锁并挂起”的需求转嫁到一个 `futex` 上，通过内核的支持正确实现了条件变量的语义。

代码片段 10.15 简化的 `futex` 接口和使用 `futex` 的条件变量

```
1 struct cond {
2     unsigned value;
3 };
4
5 void cond_wait(struct cond *cond, struct lock *mutex)
6 {
7     unsigned local = cond->value;
8     unlock(mutex);
9     futex_wait(&cond->value, local);
10    lock(mutex);
11 }
12
13 void cond_signal(struct cond *cond)
14 {
15     cond->value += 1;
16     futex_wake(&cond->value);
17 }
```

小思考

代码片段 10.15 中利用 `futex` 实现条件变量会有什么问题？

需要注意的是，代码片段 10.15 中由于 `value` 使用的类型为 `unsigned`，因此可能会发生溢出。如果 `cond_wait` 的线程放锁之后，`cond_signal` 被调用了 2^{32} 次，则 `value` 的值会溢出，导致调用 `cond_wait` 的线程丢失信号并陷入无限等待。因此，真实场景中将使用更加复杂的设计以避免溢出问题。

10.6 案例分析：微内核中的同步原语

微内核中同步原语的使用与实现与宏内核基本一致。我们可以将同步原语分为两类。第一类同步原语的实现与使用均与操作系统内核无关，如本章介绍的互斥锁和读写锁。这些同步原语在宏内核与微内核中的实现完全一致，并且可以在内核态与用户态直

接使用。

另一类同步原语则需要操作系统的支持，比如条件变量与信号量的等待操作（即 wait 操作），需要内核辅助保证一个线程能够原子地放锁并阻塞，从而避免丢失唤醒。无论是宏内核还是微内核，均需要提供对应的机制予以支持。我们在 10.5 节介绍了 Linux 中提供的 futex 机制，并展示了如何使用 futex 机制实现条件变量。本书配套的 ChCore 微内核则提供了与内核信号量实现类似的功能。顾名思义，ChCore 的内核信号量在内核中提供了信号量的功能。用户可以通过系统调用创建、等待（wait）、唤醒（signal）一个内核信号量。在创建时，内核会返回一个对应该内核信号量的编号，用户将使用该编号来对该内核信号量进行操作。由于信号量的所有操作都发生在内核态，内核信号量可轻松实现原子放锁并阻塞的功能，避免丢失唤醒。具体而言，内核中的每一个信号量都搭配了一把互斥锁。当一个线程调用 wait 时，内核将先获取该互斥锁，判断是否需要阻塞等待；在需要阻塞等待时，将该线程设置为阻塞状态，放入信号量的等待队列完成阻塞，最后再释放该信号量的互斥锁。由于阻塞操作是在临界区内完成的，不会有其他线程在阻塞之前执行唤醒操作，因此不会出现丢失唤醒的问题。通过使用内核信号量，用户态可以非常方便地实现阻塞互斥锁、条件变量与信号量。

10.7 思考题

1. 代码片段 10.16 是一个简单的用 C 语言实现的栈。小明为了确保线程安全，将中间一些操作替换成了原子操作。小明这样实现正确吗？如果正确，请说明理由。如果不正确，请指出导致不正确的具体情况。

代码片段 10.16 栈操作

```

1 typedef struct Node {
2     struct Node *next;
3     int value;
4 } Node;
5
6 void push(Node **top_ptr, Node *n)
7 {
8     Node *top = *top_ptr;
9     while (atomic_CAS(top_ptr, top, n) != top)
10         top = *top_ptr;
11     n->next = top;
12 }
13
14 Node *pop(Node **top_ptr)
15 {
16     if (*top_ptr == NULL)
17         return NULL;
18 
```